

It's the Kernel's Fault!

Custom Page Fault Handling With `bpf_fault`

Tal Zussman
Columbia University

Riju Dey
Columbia University

Hasan Zengin
Columbia University

Yiming Fang
Columbia University

David Hildenbrand
Technical University of Munich and
Arm Germany GmbH

Asaf Cidon
Columbia University

Abstract

Page faults, which occur when a program accesses a virtual memory page that is not mapped to physical memory, are traditionally handled by the operating system. However, many applications benefit from running custom page fault handling logic. For example, some applications may seek to prefill newly faulted pages with content, or intercept writes in order to make a copy of the original contents. Linux's `userfaultfd` interface enables some of these use cases by offloading fault handling to userspace. Unfortunately, it suffers from significant limitations, namely high overhead, poor scalability, and a design that precludes its use in libraries.

We present `bpf_fault`, a framework that allows applications to run page fault handlers directly within the Linux kernel using eBPF. By eliminating the overheads and complexity associated with `userfaultfd`, `bpf_fault` reduces fault latency by 2.8–6.7× and eliminates `userfaultfd`'s scalability bottleneck. We integrate `bpf_fault` with several applications, such as VM live snapshots in Firecracker and QEMU, which eliminates tail latency spikes caused by snapshots. By leveraging `bpf_fault`, we also design a novel lazy dynamic linking mechanism for Linux that defers relocations to fault time, a use case impossible with `userfaultfd`, which reduces dirty memory usage of widely used applications like Chrome and Clang by up to 50%. Together, these results demonstrate that eBPF-based fault handling can improve performance and reduce resource usage in widely deployed applications.

To find fault is easy; to do better may be difficult.

—Plutarch

CCS Concepts: • Software and its engineering → Memory management.

Keywords: Operating systems, eBPF, page faults



This work is licensed under a Creative Commons Attribution 4.0 International License.

SOSP '26, Prague, Czech Republic

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2585-2/26/09

<https://doi.org/10.1145/3830418.3843896>

ACM Reference Format:

Tal Zussman, Riju Dey, Hasan Zengin, Yiming Fang, David Hildenbrand, and Asaf Cidon. 2026. It's the Kernel's Fault! Custom Page Fault Handling With `bpf_fault`. In *Symposium on Operating Systems Principles (SOSP '26), September 29–October 2, 2026, Prague, Czech Republic*. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3830418.3843896>

1 Introduction

Page faults are a fundamental mechanism in operating systems' virtual memory architecture, enabling core features like demand paging and copy-on-write. While page faults have historically been handled entirely in-kernel, there are many scenarios where applications may benefit from customizing fault handling. Extensible operating systems in the '80s and '90s investigated the feasibility and applicability of such customization [34, 51, 78, 81]. Appel and Li [9] cataloged a wide range of applications, such as garbage collection, object stores, and distributed shared memory, which benefit from the ability to intercept page faults. SPIN [12] and Mach [1, 71] implemented user-defined fault handling as a native OS mechanism. However, these operating systems did not achieve widespread adoption, and the idea of custom page fault handling faded from the spotlight for decades.

As the memory subsystem of operating systems has become more complex, with technologies such as virtual machines and tiered memory [19, 42, 48, 72, 74], the potential use cases for such customization have only increased. In 2015, Linux 4.3 introduced the `userfaultfd` syscall (`uffd`), a mechanism for offloading page fault handling to userspace [10]. Originally designed to enable post-copy live migration of virtual machines, `uffd` has since been adopted across a wide variety of applications, including tiered-memory management [49, 66, 67, 72], far-memory systems [38, 89], VM and container snapshotting and restoration [2, 29, 60, 85], and the Android Runtime garbage collector [36].

While `uffd` introduced custom fault handling to Linux, its design imposes significant limitations on both performance and adoption. In particular, `uffd` handlers run entirely in userspace, typically in a dedicated thread, and require a context switch from the faulting thread to the handler thread. `uffd` provides no mechanism for in-kernel decision-making:

even trivial decisions, such as whether to zero a page, require a full round-trip to userspace, incurring significant overheads. In some cases, the system may experience millions of page faults per second, in which case `uffd` itself becomes a performance bottleneck. Indeed, this has long been an issue with userspace fault handling: SPIN, which let applications register custom page fault handlers within the kernel, noted that “a longstanding problem with fault-based strategies has been the overhead of handling a page fault in an application” [12, 39]. We discuss additional limitations, such as scalability, applicability, and security, in §3.

We seek to allow applications to run custom fault handling logic directly in the Linux kernel without sacrificing performance. We design `bpf_fault`, a framework that allows applications to attach eBPF programs [32] to the kernel’s page fault path without further modifications to the kernel, yielding a simpler and more scalable alternative to `uffd`. Given the advent of eBPF-based kernel extensibility frameworks such as `sched_ext` [21] and `cache_ext` [99], we seek to realize that vision and demonstrate the utility of eBPF for custom page fault handling. Indeed, in conversations with the `uffd` developers, they expressed that an eBPF-based approach such as `bpf_fault`’s may be a better fit than `uffd`’s fully userspace-based architecture [11].

`bpf_fault` runs fault handlers as eBPF `struct_ops` programs, invoked in-kernel in the faulting thread’s context. This eliminates the context switches, handler thread, and extra data copies inherent to `uffd`. `bpf_fault` supports handling the same fault types as `uffd` and extends fault handling to file-backed memory, which `uffd` does not support. Implementing this required integrating eBPF into the kernel’s page fault path, including allowing handlers to safely modify faulting pages and ensuring correct handler lifecycle management across `fork()`. `bpf_fault` also replaces `uffd`’s asynchronous event API with in-kernel callbacks, eliminating the need for a monitor thread. Because handlers are eBPF programs, the verifier guarantees bounded execution, mitigating the TOC-TOU exploits that have plagued `uffd` [23, 44, 64].

`bpf_fault` reduces fault latency by 2.8–6.7× compared to `uffd`, adds only 1.4× overhead compared to the kernel’s default fault path, and scales linearly with the number of faulting threads. We evaluate `bpf_fault` on four applications. For VM live snapshots, we integrate `bpf_fault` with Firecracker [2] and QEMU, where in-kernel fault handling eliminates the tail latency spikes caused by `uffd`. We also design a lazy dynamic linking mechanism for Linux that defers relative relocations to fault time—a use case that is impossible to support with `uffd`. On Google Chrome, this reduces dirty memory by 50% (95 MB) across real workloads and initialization time by up to 40%, outperforming Chrome’s zygote process (a Chrome-specific optimization) by 23% in physical dirty memory, using a fully generic mechanism. We also push the compaction phase of a JVM garbage collector into the kernel: eBPF fault handlers materialize the compacted

heap on demand, cutting GC pauses by up to 42% while exceeding stop-the-world throughput. Finally, we redesign an electric fence memory debugger with `bpf_fault`, improving performance by up to 4.4× over `uffd`’s handler mode while always correctly handling kernel-originated faults.

We have open-sourced `bpf_fault` and all integrations here: <https://github.com/bpf-fault/bpf-fault>. Our major contributions are:

- `bpf_fault`, a general-purpose framework for custom page fault handling with eBPF in Linux.
- Integration with VM live snapshotting in Firecracker and QEMU, reducing average latency during snapshots by up to two orders of magnitude.
- A novel lazy dynamic linking mechanism for Linux, reducing startup time and dirty memory usage.
- A comprehensive evaluation of `bpf_fault` across microbenchmarks and real-world applications.

2 Background

In this section, we describe Linux virtual memory, focusing on the mechanisms relevant to page fault handling and `uffd`, based on Linux v6.17. We also provide an overview of eBPF.

2.1 Virtual Memory

The kernel manages a process’s virtual address space through two key abstractions: virtual memory areas and page tables.

2.1.1 Virtual Memory Areas. An address space is defined by a set of virtual memory areas (VMAs). Each VMA is a contiguous address range with uniform properties: the same permissions (read, write, execute), flags, backing store (e.g., anonymous memory or a file), `uffd` handler, and more. On operations that change a property of an address range, such as `mprotect()` or `uffd` registration, the kernel splits and merges VMAs as necessary to maintain uniform properties. VMAs are stored in a tree indexed by virtual address for efficient lookup [75]. Non-anonymous VMAs implement a `fault()` callback that is invoked on page faults. For file-backed VMAs, this typically reads the page from storage.

2.1.2 Page Tables. To enable translation of virtual to physical addresses, the kernel maintains page tables for each process. Each page table entry (PTE) contains a page frame number and various bits (e.g., present, writable). On modern architectures, several PTE bits are ignored by the hardware, and used by Linux to store kernel metadata. For example, on x86-64, one such bit marks a page as `uffd` write-protected. When the processor encounters a PTE that is non-present or has insufficient permissions, it raises a page fault exception.

2.1.3 Page Faults. When a page fault occurs, the processor traps into the kernel. The kernel looks up the faulting address in the process’s VMA tree. If no VMA is found, or if the access violates the VMA’s permissions, the kernel raises a

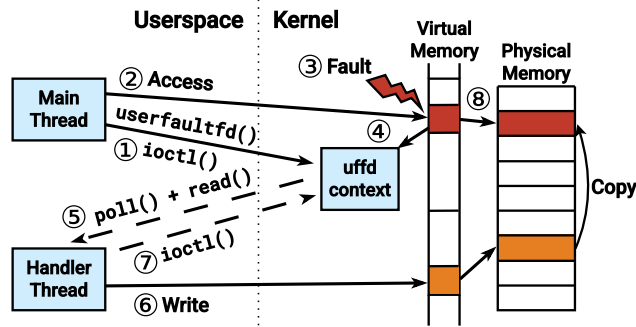


Figure 1. Handling page faults with userfaultfd.

SIGSEGV signal. Otherwise, the kernel walks the page tables, allocating intermediate levels if necessary. When it arrives at the PTE, the kernel checks the PTE's state to determine if one of the following fault types applies.

Major and minor faults. If the PTE is not present, the fault is either *major* or *minor*. Major faults occur when the page is not in memory. The kernel either allocates a new zeroed page or reads it from storage. A minor fault occurs when the page is in memory but not mapped in the process's address space. The kernel sets the PTE to point to the page.

Protection faults. If the PTE is present but the access violates its permissions, this is a *protection fault*. The most common case is a write to a non-writable page (write-protect fault), often caused by copy-on-write (COW) [41], but also used by the kernel for memory access tracing [87].

2.2 userfaultfd()

The `userfaultfd()` system call was introduced in Linux 4.3 to allow userspace to intercept page faults by offloading them to a fault-handling thread. As shown in Figure 1, a process creates a `uffd` file descriptor by calling `userfaultfd()`, and registers an address range with the `UFFDIO_REGISTER` `ioctl` (1). The application creates a fault-handling thread. When a thread triggers a fault in a registered region (2) (3), the kernel blocks the faulting thread and enqueues a fault message onto the `uffd` file descriptor (4). The handler thread, which polls the file descriptor, is woken up and reads the fault message (5). For a *missing* fault (an access to a not-yet-populated page), it prepares the page data (6), and resolves the fault by issuing an `ioctl`: for example, `UFFDIO_COPY` copies data from userspace into the page (7). The kernel then installs a PTE (8). The handler thread goes back to polling, and the faulting thread is woken up and retries the access. The various kernel crossings and context switches lead to high overhead (§3.1).

2.2.1 Fault Handling Modes. As of Linux 6.17, `uffd` supports three types of faults: missing, write-protect, and minor.

Missing mode. This is the most commonly used mode, covering live migration [10, 22], far-memory systems [38, 89] and lazy snapshot restore [2], in which the application must supply page content on first access. On a fault, the kernel wakes the handler thread instead of allocating a zeroed page.

Write-protect (WP) mode. This mode intercepts writes to pages made read-only by the `UFFDIO_WRITEPROTECT` `ioctl`, and is used for live snapshots [29, 60], garbage collection [36], and tiered memory [72]. Unlike `mprotect()`, which modifies VMA permissions, this `ioctl` operates on PTEs, protecting individual pages without splitting the VMA. The fault handler resolves the fault by making the page writable again.

Minor mode. `uffd` supports minor faults only for `shmem` and `hugetlbfs`. The handler issues a `UFFDIO_CONTINUE`, which installs a PTE pointing to the existing page in the page cache and wakes the faulting thread. If the handler wants to modify page data before installing the PTE, it must map the file separately in a non-`uffd` mapping, and modify it there.

2.2.2 SIGBUS Mode. This mode delivers a `SIGBUS` signal to the faulting thread without handling the fault, turning `uffd` into a fault *notification* mechanism. This is useful for applications that don't want a handler thread or don't want the kernel to automatically allocate pages on access. A memory debugger, for example, wants to be notified on an out-of-bounds access, not to be granted a fresh page (§5.4). However, signal handlers face `async-signal-safety` restrictions and contention on per-process signal data structures [49].

2.2.3 Non-Cooperative Mode and Events. `uffd` supports transferring fault-handling responsibilities to a *monitor* process, which handles faults on behalf of a target process. This mode is essential for checkpoint/restore (CRIU), where the restored process cannot be modified [29]. In *non-cooperative mode*, the monitor tracks changes to the target's address space via asynchronous notifications for operations like `fork()` and `mremap()`. When the target calls `fork()`, the kernel delivers a new `uffd` file descriptor to the monitor.

2.3 Signal Handling

Before `uffd`, userspace intercepted page faults by marking pages inaccessible with `mprotect()` and handling the resulting `SIGSEGV`. This technique is still used in some cases and is the only option for custom fault handling on other OSes. However, it suffers from VMA fragmentation (`mprotect` may split VMAs), inability to distinguish fault types, restriction to the faulting process (no non-cooperative mode), and difficulty handling faults triggered by system calls (e.g., `read()`).

2.4 eBPF

eBPF allows userspace programs to run safely in the kernel. The kernel *verifies* eBPF programs before execution, ensuring that they are bounded and memory-safe. While traditional use cases for eBPF include networking [43, 95, 96], observability [46], and security [50, 62], recent work has applied eBPF to scheduling [21, 47, 52], I/O acceleration [16, 91, 94], memory management [17, 26, 90, 99], and more [69, 92, 97].

Maps and arenas. eBPF maps enable sharing data between eBPF and userspace, and include hash maps and ring

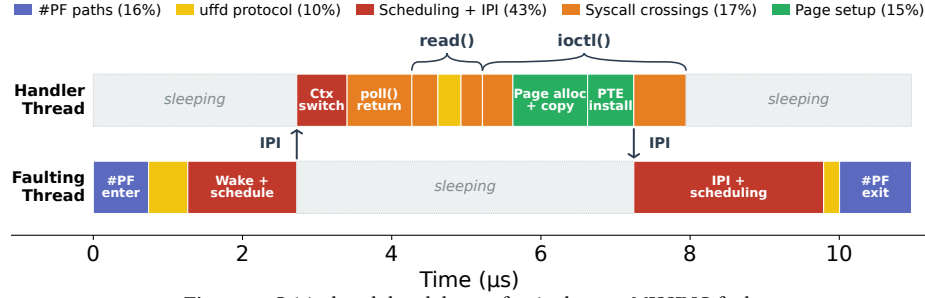
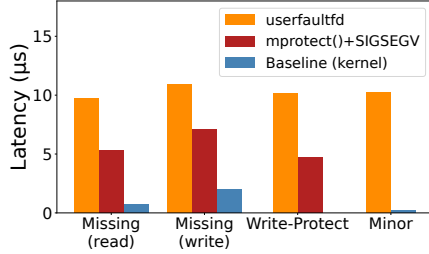
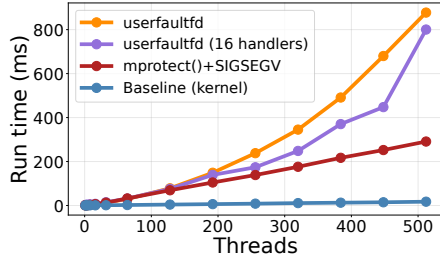


Figure 2. Critical-path breakdown of a single uffd MISSING fault.



(a) Average latency of different page fault handling mechanisms.



(b) Scalability benchmark run time under increasing load.

Figure 3. Userfaultfd performance limitations.

buffers. BPF arenas [5, 6] provide a shared memory region between eBPF programs and userspace, backed by an `mmap()`-able virtual file. They allow for dynamic memory allocation and implementation of complex data structures.

struct_ops. We use the eBPF `struct_ops` interface to implement `bpf_fault`. `struct_ops` allows userspace to attach a set of eBPF programs to the kernel as a struct of function callbacks, which the kernel invokes [57]. This interface has been used to implement custom schedulers in `sched_ext` [21] and caching policies in `cache_ext` [99], along with many other use cases [24, 26, 27, 33, 45, 56, 92]. The `struct_ops` interface uses an eBPF map. An eBPF link binds this map to the kernel and manages the lifecycle of the attached programs.

3 Motivation

uffd’s design imposes significant limitations, which we characterize and analyze in this section.

3.1 Overhead

Page faults for uffd-registered memory are subject to significant overhead due to three main factors: (1) context switches and inter-processor interrupts (IPIs), (2) kernel-user crossings, and (3) uffd-specific work. We instrument the page

fault path while triggering anonymous memory page faults in a loop. We break down how time is spent when handling MISSING faults with uffd and `UFFDIO_COPY` in Figure 2. We run the threads on two separate CPUs, preventing contention, so the measured overhead is the best case for uffd.

While the initial and final kernel crossings due to the fault trap are unavoidable (16%), there are four more crossings in the critical path (17%), where the handler thread returns from `poll()` and calls `read()` and `ioctl()`. The return from `ioctl()` overlaps with the faulting thread, as it is woken up as part of the `ioctl()`, and is excluded from the runtime. We also omit the `poll()` call in the handler thread from the figure, as it is not in the critical path. In this example, userspace (<1%) does not copy data. Context switches and scheduling take up 43%, while uffd-specific processing, such as enqueueing and reading the fault message, takes up 10%, for a total of 70% spent on uffd-inflicted overhead.

This overhead translates into significantly slower page faults compared to regular faults (Figure 3a). On a missing read fault, uffd takes 10.2 μs per fault, compared to 0.75 μs for the baseline (13.5× slower). For missing read faults, the kernel uses the zero page, as opposed to write faults, which allocate a page. Accordingly, we used uffd `UFFDIO_ZERO` and `UFFDIO_COPY` for reads and writes, respectively, to minimize differences. We used anonymous memory for missing and WP faults, and `shmem` for minor faults. Surprisingly, we find signal handling to be faster than uffd for both missing and WP faults, contrary to the documentation. However, uffd provides several benefits compared to signal handling (§2.3). We note that WP faults do not have an exact baseline mechanism, and minor faults lack an `mprotect()` analogue.

uffd also imposes excess data copying. When a handler must supply page content (e.g., reading a file), the data is first read into a userspace buffer, then copied into the kernel via `UFFDIO_COPY`. This extra copy is inherent to uffd’s design, as the handler has no direct access to the kernel page. Figure 2 does not include this cost. Handlers that copy data would see an even larger fraction spent on uffd overhead.

3.2 Scalability

When running a multi-threaded workload, this per-fault overhead compounds and the single fault handler thread becomes a serializing bottleneck. To evaluate this, we set

up a thread pool, where each thread receives 64 pages of anonymous memory and triggers page faults by writing data to each of them. We measure the total time it takes to run this benchmark across a varying number of threads (Figure 3b). While the baseline goes from 0.3 ms with 1 thread to 17 ms with 512 threads (57× longer), `uffd` with a single handler goes from 1.4 ms to 863 ms (616× longer), scaling ~10× worse. Applications can mitigate this by running multiple fault-handling threads, but this adds complexity and CPU overhead without significantly improving scalability. Even with 16 handler threads, performance only improves by 36% to 557 ms at 512 threads, while consuming 5× more CPU time than the single-handler case.

3.3 Applicability

`uffd` requires a dedicated fault-handling thread, limiting which systems can use it. Applications that fork may not be able to use `uffd` correctly, as forking in a multi-threaded environment only clones the thread that calls `fork()`, not the fault-handling thread, potentially leading to incorrect behavior. This can be mitigated by using `atfork` handlers, as `efency` does [77], but this does not work for applications that bypass `libc`, such as Chrome, which calls `clone()` directly. Non-cooperative mode can help, but that may turn the monitor into a bottleneck. In practice, this prevents `uffd` usage in libraries such as the dynamic linker (§5.2) or non-runtime garbage collectors, which cannot prevent applications from forking or may need to be transparent. Additionally, `uffd`'s missing mode does not support file-backed memory, which is how executables are commonly mapped (§5.2). Finally, `uffd` handlers cannot access kernel state directly. For example, tiered memory systems could benefit from inspecting PTE bits to support informed page placement [72].

3.4 Security

`uffd` has been responsible for several vulnerabilities [23, 44, 59, 64, 65]. The majority of these take advantage of the fact that `uffd` can indefinitely block kernel execution, enabling TOCTOU exploits. While mitigations have been developed [23, 25], they have further limited `uffd`'s functionality by blocking unprivileged `uffd` by default or preventing handling of faults originating in-kernel. Indeed, popular container runtimes like Docker and Podman have blocked `uffd` by default due to security concerns [20, 30].

4 Design and Implementation

In this section, we present `bpf_fault`'s design and implementation, and discuss how it addresses `uffd`'s deficiencies. At a high level, `bpf_fault` shifts the role of the fault-handling thread into the kernel in the form of an eBPF program (Figure 4). An application attaches a `struct_ops` program to a virtual address range, and the kernel invokes it directly on faults. This eliminates much of the overhead associated with

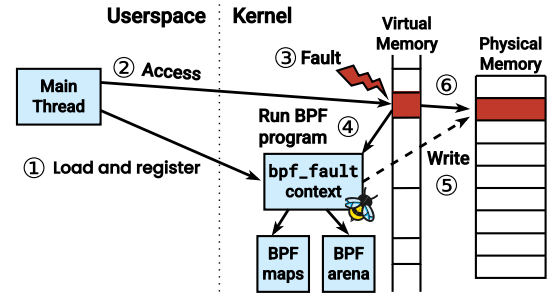


Figure 4. `bpf_fault`'s fault-handling path.

`uffd`, as there are no context switches or extra kernel-user crossings. `bpf_fault` supports anonymous and `shmem` missing faults, WP faults, and `shmem` minor faults. It also handles missing and minor faults for file-backed mappings, which `uffd` does not support: when a page is in the page cache, the handler receives it filled with the existing data. Otherwise, it is brought into the page cache.

`bpf_fault` is designed to not add significant overhead or memory usage to non-`bpf_fault` workloads. For example, it does not increase the size of any kernel memory management data structures. `bpf_fault` also enables zero-copy custom page fault handling by writing directly to the newly allocated page. This required enhancing the eBPF interface to support writing to memory in a safe and controlled manner.

Fault handling is uniquely constrained as a `struct_ops` target. Unlike scheduling or caching, where a suboptimal policy degrades performance, a fault handler runs on the critical path of every memory access: a bug causes a crash, and a slow handler directly blocks execution. Handlers cannot sleep by default, as doing so is a security risk (§4.7). However, `bpf_fault` supports opt-in sleepable handlers for use cases that require blocking. Fork creates lifecycle challenges: a *fault context* (a kernel object that ties a handler to the memory it manages) may span multiple VMAs, but the kernel duplicates VMAs individually, requiring fault context deduplication. More broadly, `bpf_fault` enables userspace to implement kernel-level mechanisms that other operating systems build natively (e.g., lazy relocation), without modifying the kernel. The kernel changes comprise ~2,500 LoC: 1,750 in memory management, and 750 in eBPF and libbpf.

4.1 eBPF Interface

The `bpf_fault struct_ops` type defines a callback for each fault type. Figure 5 shows the callbacks and the context struct passed to them. The context contains the fault type and both the page-aligned and exact faulting address. For missing faults, the second argument is a freshly allocated zeroed page the eBPF program may write to directly. For minor faults, it contains the existing contents of the faulted page. For WP faults, it is a read-only mapping of the existing page. A return value of zero instructs the kernel to install the page into the process's page tables. A non-zero return delivers `SIGBUS` to the faulting thread. This deviates from `uffd`, where a handler

```

/* bpf(BPF_LINK_FAULT_OPS_CMD) commands */
#define BPF_FAULT_WP_ENABLE    (1U << 0)
#define BPF_FAULT_REGISTER    (1U << 1)
#define BPF_FAULT_UNREGISTER  (1U << 2)
#define BPF_FAULT_CLAIM       (1U << 3)

struct bpf_fault_ops_ctx {
    unsigned long address;    /* page-aligned */
    unsigned long real_address; /* exact */
    __u8 fault_type;
};

struct fault_ops {
    int (*handle_missing_fault)(
        struct bpf_fault_ops_ctx *ctx,
        unsigned char *page);
    int (*handle_minor_fault)(...);
    int (*handle_wp_fault)(...);
    int (*handle_fork)(struct bpf_fault_fork_ctx *ctx);
    ...
};

```

Figure 5. bpf_fault's interface.

that cannot resolve a fault has no mechanism to signal failure; the faulting thread simply blocks indefinitely.

4.1.1 Writing to Memory. The eBPF verifier enforces the different permissions for the callbacks' page argument: custom verifier ops mark the page parameter as PTR_TO_MEM with PAGE_SIZE bounds for missing and minor faults, and MEM_RDONLY for WP faults. We implemented eBPF verifier support for bounded writes to PTR_TO_MEM arguments with minimal changes to the verifier. Similar functionality already exists in eBPF for writing to packets, so we do not violate verifier security guarantees. Some eBPF I/O acceleration systems have implemented similar support [91, 94].

4.2 Registration

Link creation. An *eBPF link* is a kernel object representing the attachment of an eBPF program. In practice, an application loads its handlers and attaches the struct_ops map to an address range with a single libbpf call, which issues the bpf() link-creation command. The kernel allocates the link and a refcounted *fault context*, which ties together the relevant memory regions and the struct_ops. It then walks each VMA in the target range under a lock and validates them: each VMA must be faultable, must not belong to a different fault context, and more. The kernel marks each VMA with the VM_BPF_FAULT flag, stores a pointer to the fault context in it, and splits and merges VMAs as necessary. If any VMA fails validation, the entire registration fails.

Multi-region support. After bpf_fault link creation, the application may add or remove additional disjoint address ranges on the same link, allowing a single link to manage

non-contiguous regions. This is facilitated by each VMA storing a pointer to its owning fault context. Removing a region is a two-phase operation: a validation pass to ensure every VMA in the range belongs to this fault context, followed by a modification pass that clears the VMA flags, removes the fault context pointer, and clears WP markers if relevant. Applications issue post-creation operations, such as registering a region, through a new, bpf_fault-specific bpf() syscall command, which takes a link, flags, and an address range.

4.3 Fault Path

When a page fault reaches a VMA registered with bpf_fault, the kernel diverts into the bpf_fault fault handler. All supported fault types follow a common two-phase protocol. This protocol is similar to the uffd protocol, but without the complexity of an event queue and separate thread.

Phase 1: Release and invoke. The handler releases the (per-VMA) fault lock, allocates a page if necessary, and invokes the eBPF program. Because the handler runs in the faulting thread's context, there is no handler thread serializing faults, which yields better scalability than uffd (§3.2).

Phase 2: Re-lock and install. After the eBPF program returns, the handler re-acquires a VMA lock, checks that it is still valid, and installs the PTE. If the program returns an error, the handler sends a SIGBUS to the faulting thread.

4.3.1 Fault Types. bpf_fault hooks into the anonymous page fault handler in the read- and write-fault paths. bpf_fault intercepts shmем faults at two points: when the page cache already contains a folio (minor fault) and when it does not (missing fault). In the former, the kernel passes the folio from the page cache to the minor fault callback. For file-backed VMAs, the kernel populates the page cache before calling the handler. The handler itself does not perform I/O.

WP faults are triggered when a PTE carries the WP marker on a bpf_fault VMA. The handler maps the page read-only, calls the WP callback, and clears the marker if the program permits the write. Userspace WP-marks address ranges via the new bpf() command with a WP flag, using the same logic as the UFFDIO_WRITEPROTECT ioctl.

4.4 Interactions with MM Subsystems

uffd interactions. bpf_fault shares some existing uffd infrastructure rather than introducing entirely parallel mechanisms. This enables a low code and memory footprint. At the VMA level, bpf_fault stores fault contexts in the existing vm_userfault_ctx field via a union. bpf_fault adds no new fields to VMAs, and therefore no per-VMA memory overhead. The VM_BPF_FAULT flag differentiates bpf_fault VMAs from uffd's. For WP, bpf_fault reuses the existing uffd WP PTE bits rather than allocating new PTE software bits, which are a scarce resource on x86-64. bpf_fault uses the same page table routines as uffd's WRITEPROTECT, COPY, and ZERO ioctls, in order to avoid duplicating complex PTE

and TLB logic. Both `bpf_fault` and `uffd` store their context in the same field, so a VMA cannot be registered with both, which we consider a reasonable restriction.

Other subsystems. Once a page is installed, it is a normal page from the kernel's perspective. Page migration, compaction, and KSM can operate on `bpf_fault` pages normally, as they preserve page contents. Swap-in similarly restores contents directly. If a page is discarded (e.g., `MADV_DONTNEED`), the next access triggers a new fault and the handler is invoked again; handlers should therefore be idempotent.

Suppressed optimizations. `bpf_fault`, like `uffd`, disables several kernel optimizations that batch or coalesce page operations, as they are incompatible with per-page fault handling. Transparent huge pages are disabled at both fault and collapse time, when `khugepaged` merges base pages into a huge page. Batched `shmem` folio allocation is similarly disabled. Fault-around is suppressed to prevent the kernel from mapping neighboring pages without invoking the handler. These are fundamental requirements of custom fault handling: every page installation must pass through the handler. `bpf_fault` disables these optimizations only for registered VMAs. The rest of the process is unaffected.

4.5 Event Handling

Beyond page faults, applications trigger virtual memory events (e.g., `fork()`, `munmap()`) that `uffd` exposes via its message protocol (§2.2.3). Each `uffd` event wakes the monitor, adding scheduling overhead. `bpf_fault` eliminates this overhead for events as it does for faults: each event becomes a callback in the `fault_ops` structure, invoked in-kernel. Supporting a new event requires only calling the callback in the relevant function. For non-cooperative mode, this eliminates the need for a monitor, as events are handled non-intrusively. Use cases that require notifying an external process can do so from within eBPF via ring buffers or signals.

4.6 Fork Inheritance

When a process using `uffd` forks, it must either re-register its regions or, in non-cooperative mode, the monitor must poll the new file descriptor. With `bpf_fault`, when a process using the `BPF_FAULT_INHERIT` registration flag forks, the child automatically receives a lightweight inherited link: a minimal allocation that simply holds a reference to the parent's `struct_ops`. This avoids link setup in the fork fast path. If the child later needs a full link (e.g., to register a region), it issues a `BPF_FAULT_CLAIM` command through the `bpf()` syscall, which promotes the lightweight link.

During fork, the kernel duplicates VMAs one at a time. Since a single fault context may span multiple VMAs, this would wrongly allocate a new context for each VMA. A deduplication list solves this: the first VMA encountered for a given context allocates a child context and appends it to

the list. Subsequent VMAs sharing the same context find the child via list lookup. `uffd` implements a similar mechanism.

4.7 Security

Since the fault handler is an eBPF program, it is subject to the verifier's checks at load time: bounded loops and provably safe memory accesses. As an eBPF framework, `bpf_fault` requires `CAP_BPF` to load eBPF programs. `sched_ext` and `ghOst` [21, 47] use a privileged program loader to mitigate this. Additionally, on modern systems, unprivileged `uffd` is disabled by default, so this does not affect relative usability.

By default, fault handlers are *non-sleepable*: the handler executes atomically and cannot be delayed. This closes off the most common `uffd` vulnerability pattern, in which an attacker triggers a page fault from kernel code (e.g., by calling `read()` with a not-yet-faulted address) while the userspace handler blocks indefinitely. This can cause kernel execution to be frozen in a known state, widening the timing window for TOCTOU and race-condition exploits [23, 59]. This is partially why unprivileged `uffd` is now disabled by default [23].

`bpf_fault` supports opt-in sleepable handlers for use cases that require blocking operations. Sleepable handlers reintroduce the ability to delay the faulting thread, but mounting a `uffd`-style attack with them is strictly harder: the verifier rejects unbounded loops, and the blocking operations available to eBPF, such as memory allocation, are themselves bounded and cannot wait on an attacker-controlled event. Blocking is also explicit and per-handler, unlike `uffd`, where any handler may block at any time; most handlers in our application integrations (§5) are non-sleepable.

Sleepable handlers also do not create new denial-of-service opportunities. A handler executes only on faults in registered regions, so a slow handler delays only the process that installed it, not other processes or the kernel. `bpf_fault` does not support cross-process registration.

4.8 Limitations

`bpf_fault` trades `uffd`'s generality for performance and safety. A `uffd` handler is ordinary userspace code: it can access all of the process's state or issue syscalls. A `bpf_fault` handler is limited by the verifier and `kfunc` availability. Processes can share state with handlers via eBPF maps and arenas, but this requires setting up shared structures.

In practice, the most significant restriction is I/O: eBPF programs cannot currently write to files or access the network. Our VM snapshot integration works around the former by streaming pages to a userspace writer (§5.1), but the latter prevents use cases such as post-copy live VM migration, where the handler reads data from the network. These gaps are closing as eBPF evolves. `Dynptrs` provide access to variable-sized memory [86], and a recent proposal supports sending UDP packets from eBPF, with TCP support under discussion [7]. Supporting socket reads would enable an eBPF handler to implement post-copy migration in-kernel.

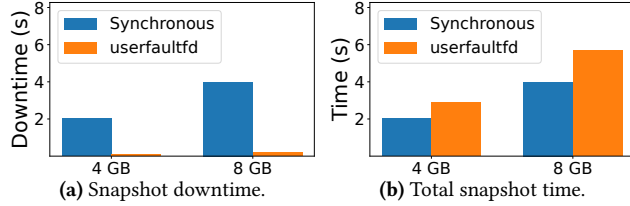


Figure 6. Firecracker (4 and 8 GB) for synchronous and uffd live snapshots.

5 Applications

We demonstrate `bpf_fault`'s utility across four applications that span different fault types and usage patterns. Our primary contributions beyond the framework itself are: (1) integrating `bpf_fault` with VM live snapshotting in Firecracker and QEMU, where in-kernel WP handling significantly reduces tail latency spikes caused by `uffd`, (2) a new general-purpose lazy dynamic linking mechanism for Linux enabled by `bpf_fault`, which defers relocations to fault time, reducing startup latency and per-process memory, (3) pushing a JVM garbage collector's compaction phase into the kernel, and (4) redesigning a memory debugger using `bpf_fault`.

The Firecracker integration is 2,600 lines of Rust and based on v1.15.0. The QEMU integration is roughly 900 LoC, based on QEMU 10.2. The dynamic linker is 500 LoC in glibc 2.41 and 2,500 LoC in the eBPF fault handler and loader. The GC integration is roughly 3,500 LoC: 2,650 in MMTk in Rust, and 880 in the eBPF handlers and loader. eBPF-based memory debugging was implemented in 1,550 LoC.

5.1 Virtual Machine Live Snapshots

VM snapshots are crucial for disaster recovery, checkpointing, and migration. The simplest approach is a synchronous snapshot: pause the VM, serialize all state, dump memory to a file, and resume. This incurs downtime proportional to guest size, making it unsuitable for latency-sensitive workloads.

Live snapshots avoid this by capturing a consistent image of a running VM without pausing it for the full duration. The VMM briefly pauses vCPUs to capture device state, write-protects all guest RAM, and resumes. A background thread iterates over guest memory and saves each page to a file. When a vCPU writes to an unsaved page, a WP fault fires, blocking the faulting vCPU. The `uffd` handler thread saves the pre-image and lifts WP, and the vCPU resumes. This round trip is on the critical path of every intercepted write, causing tail latency spikes. We integrate `bpf_fault`-based live snapshots into two VMMs: Firecracker, where we implement live snapshots from scratch with both `uffd` and `bpf_fault` backends, and QEMU, where we replace the `uffd` backend of an existing upstream feature.

5.1.1 Firecracker. Firecracker [2] is a lightweight VMM powering services like AWS Lambda. Firecracker only supports synchronous snapshots, requiring the VM to be paused

Prepare	Freeze	Stream	Finalize	Total
3,485	196	1,697	208	5,586

Table 1. uffd live snapshot phases (ms) for an 8 GB Firecracker VM.

for the entire snapshot duration. We implemented live snapshots in Firecracker with both `uffd` and `bpf_fault`, reducing VM downtime by 20× compared to synchronous snapshots (Figure 6a). Both implementations follow a four-phase model: *prepare* (allocate resources, set up eBPF or `uffd`), *freeze* (pause VM, save state, enable WP, resume), *stream* (save pages while VM runs), and *finalize* (sync and clean up). A dedicated snapshot thread, spawned before Firecracker applies seccomp filters (which block thread creation), executes the phases, keeping Firecracker's event loop responsive. Table 1 shows the time spent in each phase for `uffd` live snapshots. The prepare phase dominates total time, as it includes file allocation via `fallocate()`. Freeze (during which the VM is paused) accounts for ~3.5% of total snapshot time.

Snapshots with uffd. In the prepare phase, the VMM creates a `uffd` file descriptor. After freezing, it registers each memory slot with `uffd` WP mode and write-protects all pages. On each fault, the snapshot thread saves the pre-image and removes WP, with the vCPU blocked until resolution. In between, it iterates over guest memory and saves pages.

Snapshots with bpf_fault. We replace `uffd` with an eBPF handler that runs in the faulting thread's context. On a WP fault, the handler copies the page's pre-image into an eBPF ring buffer and returns immediately, allowing the write to proceed without blocking. The handler submits pre-images with `BPF_RB_NO_WAKEUP`, suppressing per-event IPIs that otherwise account for two-thirds of snapshot thread execution. If the ring buffer is full, the snapshot fails. In practice, sizing the buffer appropriately to memory prevents this. Allowing eBPF programs to write directly to files would eliminate this constraint. Because the handler runs atomically in-kernel, there is no context switch: the vCPU resumes in mere microseconds. The prepare phase loads `bpf_fault` programs and maps the ring buffer. After freezing, the snapshot thread attaches the fault handler to each memory slot and enables WP tracking. The streaming phase interleaves ring buffer draining with a linear scan of guest memory, writing contiguous runs of unsaved pages and resolving their WP.

5.1.2 QEMU. QEMU can take synchronous snapshots, or live snapshots using KVM dirty page logging [55]: the migration thread iteratively saves dirty pages while the VM continues running, converging toward a final brief pause in which any remaining dirty pages are saved. Under heavy write workloads, however, convergence may never occur, since previously saved pages may be re-dirtied and must be re-saved. QEMU's background snapshot mode avoids this with `uffd` WP, ensuring that pages are saved exactly once, at the cost of blocking vCPUs on writes to unsaved pages. We add a `bpf_fault` backend to QEMU, with the same design as

Application	# R_REL	% Total Rel.	% Linker Time
Google Chrome	1,075,978	99.1%	78.8%
Clang	501,741	96.4%	90.6%
Deno	180,420	99.5%	85.2%
Docker	120,231	99.8%	91.6%
Node.js	56,072	52.2%	92.8%

Table 2. R_RELATIVE relocation statistics.

our Firecracker integration: the handler writes pre-images to a ring buffer, which the migration thread drains during its scan, skipping captured pages and lifting WP in batches.

5.2 Dynamic Linking

Fast application startup and low per-process memory consumption are critical for workloads that spawn many short-lived processes: AI agents invoking compilers, serverless cold starts, and mobile applications where startup latency is user-visible. Dynamic linking affects the performance of these workloads. When a dynamically linked program is executed, the OS maps the program and its shared libraries, and the dynamic linker must *relocate* each binary, patching embedded addresses to account for the actual load address. *Relative relocations* occur when a compile-time constant holds a pointer to a location within the same binary. Each requires a single computation: store the load address plus an addend into the target location. In large position-independent binaries, these account for up to 99% of all relocations [83, 84].

On Linux, the dynamic linker applies all relative relocations eagerly at load time, writing to every page that needs a relocation. Each write triggers a COW fault, dirtying the page and consuming more memory. For large position-independent binaries, applying relative relocations can take up to 79–93% of dynamic linker startup time (Table 2). Google Chrome (1M+ relocations) mitigates this through a *zygote* process that performs relocations once and then `fork()`s children that share the already-relocated pages, amortizing ~8 MB of dirty memory across child processes [83].

Other platforms have recognized the limitations of eager relocation. Windows maps libraries to the same address across processes and applies relocations lazily in-kernel, so relocated pages remain “clean” and shareable between processes [84]. Apple introduced *page-in linking* in 2022 [54], in which the kernel applies relocations lazily. This required changing the Mach-O executable format, which now embeds relocation metadata directly in the data pages, eliminating the need to consult external relocation tables. Both approaches share the same insight: pages never accessed should not incur relocation cost. On Linux, no such kernel-level mechanism exists. `uffd` is not suitable due to its high overhead and handler thread (§3.3). We use `bpf_fault` to implement this, achieving the same lazy semantics without changes to the ELF format or the kernel’s process loader.

5.2.1 Relocations with `bpf_fault`. Rather than applying all relocations eagerly at load time, we defer them to fault

time: the first access to a page triggers an eBPF handler that locates the containing segment via a per-process red-black tree and applies only the relocations for that page. We integrate this approach into the glibc dynamic linker. The linker registers each object’s writable segments with a pre-loaded `bpf_fault` handler, and skips the relocation loop. Not all relocations can be deferred: the dynamic linker reads certain pages before the fault handler is registered, such as the `.dynamic` section. These pages are already resident, so the linker eagerly relocates them during startup. Non-relative relocations are applied eagerly by the linker as before.

A single system-wide eBPF program is loaded once and pinned. The dynamic linker opens the pinned program at startup, via `bpf()` syscalls. This avoids per-process verification (~300 ms) and a libbpf dependency, which cannot be easily used in `ld.so`, as it uses `libc` (which is not yet mapped).

Segment registration. At fault time, the eBPF handler must determine which segment corresponds to the faulting address. To do so, on startup, the dynamic linker inserts each object’s relevant segments as interval nodes into a per-process red-black tree, allocated in an eBPF arena (§2.4). Each node stores a segment’s address range and a pointer to its ELF relocation table in process memory. Registration inserts S nodes (where S is the number of writable segments) via batched `BPF_PROG_RUN` calls. A hash map maps PIDs to rb-trees, allowing the single eBPF program to serve all processes. The linker frees a process’s entries at exit and clears stale entries at startup to handle PID reuse. On `fork()`, the handler clones the parent’s rbtree and updates the map. Libraries loaded via `dlopen()` are similarly supported.

Fault-time relocation. On page faults, the handler performs an $O(\log S)$ interval search of the rbtree to find the containing segment. It then binary-searches the segment’s relocation table (sorted by offset) for the page’s relocations. The handler reads relocation entries in large batches (341 entries $\approx$ 8 KB) from process memory via `bpf_probe_read_user()` into a per-CPU eBPF array map. Accessing this buffer requires briefly disabling preemption with eBPF `kfuncs`. Batching is critical to per-fault overhead: reading one entry at a time results in 500% overhead; batching entries in 8 KB batches reduces this overhead to 57%. However, because large applications touch only a fraction of their relocated pages at runtime, deferring relocations to fault time reduces total work and can improve end-to-end performance, as we show in §6.3. Relocations are applied directly into the kernel-provided page, which contains the file contents. The dynamic linker maps libraries as file-backed mappings, so supporting this use case required adding file-backed fault handling to `bpf_fault` (§4), which `uffd` does not support.

5.3 Garbage Collection

GC pauses are a chronic problem for latency-sensitive Java services. `bpf_fault` accelerates two mechanisms that garbage

collectors rely on: tracking writes for generational collection, and intercepting accesses to objects during concurrent compaction. We implement both on OpenJDK 21 via MMTk [13, 14], a framework for pluggable runtime GCs.

5.3.1 Dirty Tracking. Most objects die young, so generational collectors place new allocations in a small nursery region that is collected frequently, but doing so efficiently requires finding pointers from older objects into the nursery without scanning the whole heap. The standard approach is a write barrier: compiled-in code on every pointer store that records the modified address. Write-protecting old pages instead replaces per-store work with one fault per written page, but the cost of `mprotect()` or `uffd` faults has kept this approach from being viable. We replace MMTk GenImmix’s barrier with page dirty tracking: the first write to a clean page faults, and the eBPF handler marks the page in a bitmap shared with the collector and resolves the fault in-kernel.

5.3.2 Concurrent Compaction. A compacting collector slides live objects together to eliminate fragmentation, which entails copying live data and updating every pointer to a moved object. If done stop-the-world, the pause is proportional to live data. Server JVMs avoid the pause with concurrent compactors like ZGC [88], at the cost of read barriers compiled into all generated code. Fault-based compaction instead limits interception to moved pages and needs no compiler support. Android’s ART takes the fault-based approach with `uffd` [36]: after a short pause, application threads (*mutators*) resume with the heap unmapped, a background thread compacts pages in address order, and a mutator that touches an uncompact page faults and blocks until its contents are ready. We implemented a similar design in the JVM on top of MMTk’s Compressor [53], a stop-the-world compactor. A single pause marks live objects, computes each object’s destination, and uses `mremap()` to move the old heap contents to an *alias mapping*, leaving the heap range empty and registered for missing faults. Compaction proceeds during the *window* between resume and the heap being fully rebuilt.

Compaction with `uffd`. During the window, GC workers compact each region in userspace: they copy every live object from the alias into a staging area, updating each pointer the object contains to its referent’s new address, then install each page with `UFFDIO_COPY`. A faulting mutator compacts the region it touched itself rather than waiting for the sweep. Each page then costs an extra kernel crossing beyond the fault itself.

Compaction with `bpf_fault`. With `bpf_fault`, the eBPF handler performs both halves of the compaction *in-kernel*. During the pause, userspace sets up compaction metadata in an eBPF arena: a bitmap of live words, a forwarding table containing each object’s new address, and, for each heap page, the alias offset where its contents begin. On a fault, the handler reads the live objects from the alias and packs them

into the faulting page. Pointer fields are relocated as they are copied, using the forwarding table. The kernel installs the result—copied, compacted, and relocated—as the page. GC workers trigger faults in address order, but concurrent mutators may fault as well. The post-GC heap is thus faulted into existence, with lower overhead during the window.

5.4 Memory Debugging

Memory safety bugs such as heap buffer overflows and use-after-free are among the most common security vulnerabilities in C/C++ programs. Electric fence memory debuggers detect these bugs by placing each allocation on its own page(s) with an adjacent guard page. Any out-of-bounds access immediately triggers a fault, trapping on the exact instruction rather than at a later bounds check, in contrast to canary-based approaches [28]. Unlike compile-time approaches like AddressSanitizer [79], electric fences can be used via `LD_PRELOAD`, but existing implementations impose severe overhead or sacrifice correctness.

5.4.1 Electric Fence. Efence, the original implementation [68], implements this with virtual memory protections. For each allocation, it uses `mprotect()` to make the guard page inaccessible. On `free()`, it revokes access to the entire allocation. This alternating protection pattern creates a separate VMA for each allocation, leading to degraded performance, making the tool impractical for large applications.

5.4.2 Efency. Efency [76, 77] redesigns electric fences using `uffd`. At initialization, it reserves a single large allocation via `mmap()` and registers it with `uffd` missing mode. The entire region has the same protections, resulting in a single VMA, solving efence’s fragmentation and overhead problem. Efency logically divides the region into two-page slots: a “fence” page at an even index followed by a usable page in the odd one. Fencing is enforced by the fault handler, which checks the index parity. A userspace bitmap tracks which slots are in use. Efency supports two modes:

Handler-thread mode. A dedicated `poll()` thread resolves valid faults via the `uffd` ioctls, populating the page. When a fault hits a fence page or a freed slot, the handler reports the error before terminating. This mode needs support for kernel-originating faults (e.g., triggered by `read()`), which requires root privileges or unprivileged `uffd`.

SIGBUS mode. With `uffd` in SIGBUS mode, efency eagerly populates pages on `malloc()` via `UFFDIO_COPY`, so valid accesses never fault. Invalid accesses trigger a fault, which the kernel delivers as a fatal SIGBUS. While this eliminates the handler thread, it shifts work into the allocation path and increases memory usage. Additionally, since valid pages do not trigger faults, this mode uses userspace-only `uffd`. This means that kernel-originated faults yield EFAULT rather than SIGBUS, and may go unnoticed.

Fault type	uffd		bpf_fault		Baseline	
	Avg	p99	Avg	p99	Avg	p99
Missing (anon)	11.6	16.8	2.7	7.8	2.0	7.0
Write-protect	10.8	13.9	1.6	1.8	–	–
Minor (shmem)	10.3	13.8	3.7	8.8	2.6	7.6

Table 3. Per-fault latency (μ s) for write-triggered faults.

5.4.3 Efficient Efency with `bpf_fault`. Efency forces a trade-off between correctness and performance. Handler mode correctly intercepts all faults, including those originating in the kernel, but requires a dedicated thread. SIGBUS mode eliminates the handler thread but eagerly populates pages, wasting memory, and may silently miss kernel-originated faults. `bpf_fault` resolves this trade-off: the in-kernel handler provides lazy page population without a dedicated thread, while correctly intercepting all faults. Using `bpf_fault`, we install an eBPF fault handler that computes the slot index from the fault address and checks the bitmap. We use an eBPF arena (§2.4) to store per-process allocation bitmaps. If the slot is valid, the kernel installs the page. Otherwise, it writes diagnostic metadata to an eBPF map and returns an error, raising a SIGBUS. A signal handler reads this map to print an error message before re-raising the signal. Crucially, because the handler runs in-kernel, it correctly intercepts kernel-originated faults (e.g., a `read()` into a guard page), which SIGBUS mode may silently miss as EFAULT. The arena enables efficient `fork()` support: a child process allocates a fresh bitmap region and re-attaches the `fault_ops` without reloading the eBPF program, avoiding the `atfork` workarounds that efency uses. We verified that `bpf_fault` detects buffer overflows and use-after-free in our test suite.

6 Evaluation

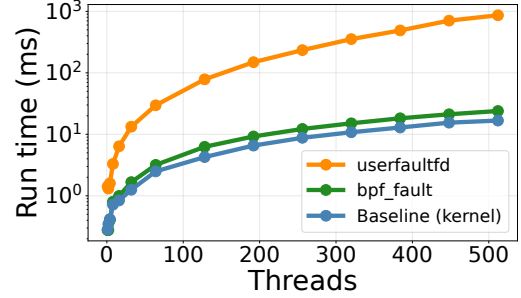
This section answers the following questions:

- Q1:** What is `bpf_fault`'s overhead relative to `uffd` and the kernel baseline, and how does it scale?
- Q2:** How does `bpf_fault` improve applications, both by replacing `uffd` and by enabling new use cases?

System configuration. We conduct our experiments on CloudLab [31] c6525-25g machines, with a 16-core AMD Rome CPU, 128 GB of memory, and a 480 GB SSD drive. We disable SMT, swap, and address space randomization for reproducibility. We run Ubuntu 24.04 with Linux v6.17.0.

6.1 Microbenchmarks

Table 3 shows per-fault latency for each fault type supported by both `bpf_fault` and `uffd`, alongside the kernel's baseline fault path. `bpf_fault` reduces average latency by 2.8–6.7 $\times$ and 99th percentile latency by up to 7.7 $\times$. WP faults benefit the most because `uffd` requires an additional round-trip to lift WP, while `bpf_fault` clears the marker in-kernel. Missing faults (4.3 $\times$) reflect the elimination of context switches

Figure 7. `uffd`, `bpf_fault`, and baseline scalability.

and extra kernel-user crossings identified in Figure 2. Minor faults see the smallest gain (2.8 $\times$) because shmem minor faults for private mappings must copy the page cache folio's contents into the new page (in order to support modifications), narrowing the gap with `uffd`. Compared to the baseline, `bpf_fault` adds 1.4 $\times$ overhead for missing write faults (2.7 μ s vs. 2.0 μ s), primarily from eBPF invocation. For unregistered memory, `bpf_fault` adds no overhead.

Figure 7 shows `bpf_fault`'s scalability alongside the baseline and `uffd` with a single handler thread, running the same experiment as in §3.2. The two lines grow with a constant $\sim 1.5\times$ gap due to eBPF overhead. At 512 threads, `bpf_fault` runs in 25 ms compared to 17 ms for the baseline and 859 ms for `uffd` (34 $\times$ slower). As shown in §3.2, even 16 `uffd` handler threads fail to meaningfully close this gap while consuming 5 $\times$ more CPU time. In contrast, `bpf_fault` requires no extra threads and uses only 1.6 $\times$ the CPU time of the baseline.

6.2 Virtual Machine Snapshots

We evaluate live snapshots in Firecracker and QEMU using Redis and Memcached with write-heavy workloads that stress the WP fault path. We run `memtier_benchmark` [73] against the guest, trigger a snapshot after 5 s of steady-state load, and record throughput and 99th percentile latency.

Figure 8 shows time-series traces for Redis on an 8 GB Firecracker VM. With the synchronous baseline (Figure 8a), throughput drops to zero for the full snapshot duration (~ 4 s) as the VM is completely paused; individual requests see up to 4 s of latency, causing request failures. With `uffd` live snapshots (Figure 8b), the VM resumes after a brief freeze (~ 195 ms), but throughput remains severely degraded for approximately 2 s during the streaming phase. As described in §5.1, each vCPU write to a protected page blocks, causing repeated 99th percentile latency spikes of up to 650 ms. With `bpf_fault` (Figure 8c), throughput dips on freeze (~ 200 ms) but recovers rapidly. The 99th percentile latency shows only a single brief spike (~ 260 ms), due to the VM pause.

Figure 9 reports throughput and average request latency measured between VM resume (after freeze) and snapshot completion (the window during which the VM is running with WP). The synchronous baseline is excluded as the VM is fully paused during this period. `bpf_fault` has 2–4 $\times$ higher

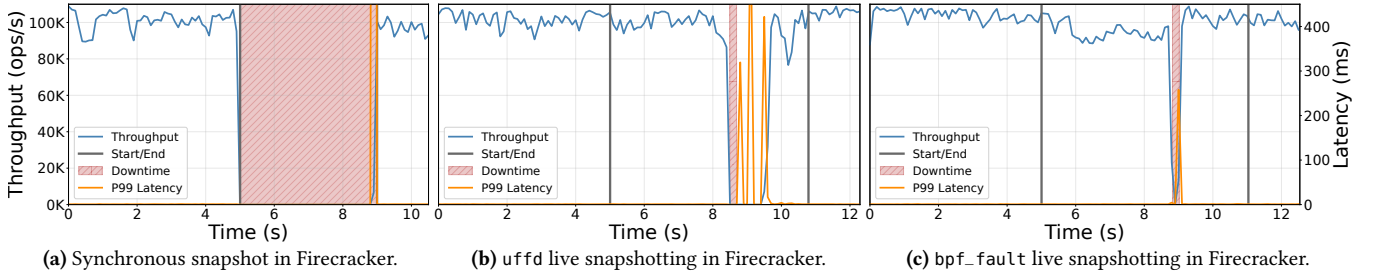


Figure 8. Redis throughput and tail latency while taking a VM snapshot.

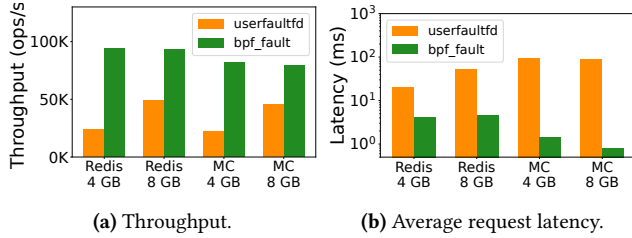


Figure 9. Firecracker throughput and latency for Redis and Memcached while the VM runs with snapshot WP active (freeze to snapshot completion).

Mode	Total		Downtime	Worst lat.
	8 GB	16 GB		
Synchronous (savevm)	2.31 s	4.36 s	2.3 s–4.4 s	2.3 s–4.4 s
Dirty tracking	2.36 s	4.37 s	134–222 ms	139–226 ms
Background (uffd)	3.33 s	6.46 s	3–4 ms	124–142 ms
Background (bpf_fault)	2.33 s	4.34 s	3–4 ms	6–7 ms

Table 4. QEMU snapshot modes with Redis.

throughput than `uffd` during snapshots (Figure 9a) and reduces average latency by 5–111 $\times$ (Figure 9b). The worst latency under `bpf_fault` is 150–260 ms (the freeze pause), compared to 1.0 s under `uffd`. The throughput gap is widest at 4 GB: the shorter streaming phase means the early period, with more write-protected pages yielding more faults, dominates the window. The improvement is due to lower per-fault overhead and the non-blocking handler design that `bpf_fault` enables (§5.1). Both `uffd` and `bpf_fault` take approximately the same total wall-clock time, but differ in the impact on the VM during the snapshot.

This technique generalizes beyond VMs: key-value stores like Redis rely on `fork()` for background checkpointing, which duplicates page tables and triggers COW faults. A `bpf_fault`-based approach could provide consistent and persistent snapshots at very low overhead.

QEMU. Table 4 compares QEMU’s snapshot modes on Redis. With `bpf_fault`, the background snapshot completes in the same time as a synchronous snapshot while reducing downtime from seconds to ~ 3 ms, and worst-case request latency during the snapshot is 20 $\times$ lower than `uffd`’s (7 ms vs. 142 ms). `uffd` instead prolongs the snapshot by 1.5 $\times$: unlike Firecracker, which has a dedicated snapshot writer thread, QEMU’s migration thread both resolves faults and streams pages, so `uffd`’s overhead stalls the snapshot itself.

6.3 Dynamic Linking

We evaluate the dynamic linker integration described in §5.2 across microbenchmarks and five real-world applications.

Microbenchmarks. Figure 10a shows startup time for libraries with 4K, 100K, and 1M relative relocations. The test program links the library but never accesses its data, isolating the relocation cost. Standard `ld.so` scales from 2.2 ms at 4K to 18.5 ms at 1M, as it eagerly writes every relocated page. With `bpf_fault`, the linker skips the relocation loop, leading to an 8.7 ms startup for 1M relocations, a 53% reduction. The remaining cost is dominated by segment registration and other linker work. The `dlopen` benchmark loads a 1M-relocation library at runtime without touching pages, completing 2.5 $\times$ faster with `bpf_fault` than baseline.

Applications. Figure 10b shows dirty memory reduction for real-world applications. Dirty pages cannot be reclaimed by the kernel without writing them to swap, and cannot be shared across processes. Reducing dirty memory directly lowers per-process physical memory consumption. Clang maintains 49–64% savings even during full -O2 compilation with linking, because execution touches only a fraction of the pages containing LLVM’s 502K relocations. Chrome’s savings remain stable at 49–53% across getting the DOM of a webpage and print-to-PDF workloads, going from 200 MB to 100 MB. Deno’s reduction narrows to 13% (from 59% at startup), as the runtime touches most of the pages covered by its 180K relocations during initialization.

In terms of runtime, Clang compilation is 7–11% faster with `bpf_fault`, as compilation touches only a fraction of relocated pages. Chrome and Deno are 4–5% faster, as application work dominates after startup. These savings compound in workloads that spawn many short-lived processes, like build systems, AI agents using tools, and serverless runtimes performing per-request cold starts, where reducing process memory usage directly improves instance density.

Startup. We benchmark the startup time of five applications using `--version`, which causes them to exit after initialization (Figure 10c). This is an important metric that has been used by both the Chromium and Node developers [37, 83]. Applications with many relocations benefit the most from deferral. Chrome, with 1M relocations, completes in 20 ms versus 36 ms (44% faster). Deno sees a 30% reduction, and Clang 16%. Docker is the one application where `bpf_fault`

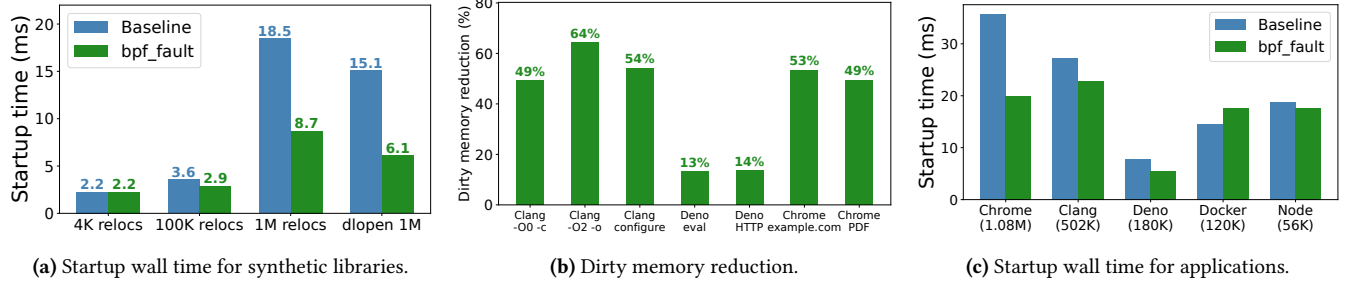


Figure 10. Dynamic linking experiments.

Workload	STW		uffd		bpf_fault	
	Time	Pause	Time	Pause	Time	Pause
h2	41.5 s	526 ms	46.9 s	382 ms	34.7 s	303 ms
pmd	4.2 s	93 ms	4.6 s	75 ms	4.3 s	73 ms
xalan	2.0 s	12 ms	2.2 s	9 ms	2.1 s	10 ms
lusearch	4.2 s	11 ms	9.1 s	10 ms	7.6 s	10 ms

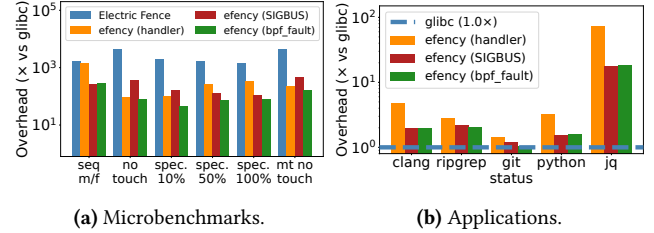
Table 5. Concurrent compaction: runtime and mean GC pause.

is slightly slower: deferring relocations moves their cost into page faults, which pay a per-fault lookup overhead (§5.2). Docker’s startup is dominated by Go runtime initialization, which touches most relocated pages immediately, so it pays this overhead without skipping much relocation work. Node, with only 56K relocations, shows minimal improvement, confirming that gains are proportional to relocation count.

Every application also sees a reduction in dirty memory at startup, because `bpf_fault` avoids dirtying pages that are never accessed. For example, Chrome saves 50%, or 95 MB, of dirty memory. Our benchmark disables Chrome’s zygote process, which normally amortizes relocation costs across processes. With `bpf_fault`, the zygote is unnecessary because relocations are deferred regardless of how a process is launched. Note that the zygote does not eliminate dirty pages: it shares them across the process tree. Measured by proportional dirty memory (PSS_Dirty), which accounts for sharing, `bpf_fault` still reduces physical dirty memory by 23% compared to the zygote (104 MB vs. 135 MB), because unreferenced pages are never dirtied. Chrome’s multi-process architecture also exercises our `fork()` support: child processes inherit the parent’s segment tree and continue to fault in relocations independently. These results demonstrate that `bpf_fault` can achieve deferred relocation semantics comparable to Windows and Apple’s page-in linking, without changes to the ELF format or the kernel’s process loader.

6.4 Garbage Collection

Dirty tracking. We run four DaCapo [15] benchmarks at a 4 GB heap under GenImmix, comparing its write barrier against dirty tracking with SIGSEGV, `uffd` WP, and `bpf_fault`. On `xalan`, whose barrier-heavy loops write few distinct pages, page-based tracking of any kind is 2.4× faster than the barrier. On the remaining benchmarks the barrier wins: SIGSEGV tracking is worse by 20–66% and `uffd` by 11–59%, but `bpf_fault` is within 5% on `lusearch` and `pmd`. `h2` is



(a) Microbenchmarks.

(b) Applications.

Figure 11. Electric fence experiments.

the worst case for all three, as it spreads writes across many pages (+46% for `bpf_fault`). `bpf_fault` reduces the overhead due to page-based dirty tracking compared to existing techniques, making it competitive when writes cluster.

Concurrent compaction. Table 5 compares the stop-the-world (STW) Compressor against the `uffd` and `bpf_fault` integrations, at heaps 1.5× each benchmark’s minimum. At these sizes, each iteration triggers hundreds of collections (~120 for `h2`), each of which compacts the entire heap. On `h2`, `bpf_fault` wins on both metrics: it reduces mean pauses by 42% below stop-the-world while running 16% faster, and beats `uffd` by 21% and 26% respectively. The `uffd` window is expensive, with GC workers copying pages for the entire window. This can delay subsequent collections: if the heap fills before the compaction finishes, the next collection must wait, which is why `uffd`’s worst-case pause (823 ms) exceeds even stop-the-world’s (757 ms). `bpf_fault`’s windows close quickly, and its worst-case pause (590 ms) is the lowest. `pmd` shows the same pattern at smaller scale: 22% lower pauses for 2% worse runtime. `xalan` and `lusearch` pause for only ~10 ms, so concurrency has little to gain, but every collection pays the window’s fixed setup cost, which at hundreds of collections outweighs the savings, leading to worse runtime. On pause-dominated heaps, `bpf_fault` improves pause times and throughput compared to stop-the-world. While it does not beat STW on every workload, `bpf_fault` makes virtual memory GC techniques viable for further experimentation, where they have previously been expensive.

6.5 Memory Debugging

We compare the three efency modes (handler, SIGBUS, and `bpf_fault`) on microbenchmarks (Figure 11a) and on real applications (Figure 11b). Results are normalized to glibc, which completes microbenchmarks 2–3 orders of magnitude faster than efency. On applications, overhead compared to

glibc is far lower (1–2× on most applications), as allocation is a fraction of total work. We omit Electric Fence from the application benchmarks: it is extremely slow, and crashes on clang and ripgrep due to kernel VMA limits.

Microbenchmarks. `seq_malloc_free` allocates, touches, and frees in a loop; `no_touch` and its multi-threaded variant, `mt_no_touch`, allocate and free without dereferencing; and the speculative benchmarks allocate 1024 objects in bulk but only touch a random 10%, 50%, or 100%, modeling patterns like hash tables and connection pools where slots go unused. When every allocation is used (`seq_malloc_free`), SIGBUS slightly outperforms `bpf_fault`, as eager population avoids faults. However, `bpf_fault` dominates the other benchmarks. On `no_touch`, `bpf_fault` is 4.7× faster than SIGBUS, which populates unused pages. With threads, both modes slow and the gap narrows to 2.7×, with `bpf_fault` remaining fastest. On speculative benchmarks, `bpf_fault` scales with utilization (3.6× faster than SIGBUS at 10%, 1.4× at 100%). SIGBUS does not match this scaling because it cannot intercept kernel-originating faults (which return `-EFAULT` rather than delivering a signal), forcing it to eagerly populate pages to remain correct. In contrast, `bpf_fault` resolves faults transparently in the kernel, enabling lazy population without this tradeoff. Handler mode is consistently the slowest (up to 4.4× slower than `bpf_fault`).

Applications. We chose applications that span the allocation behaviors electric fences must handle: clang allocates huge volumes of small, short-lived objects; ripgrep is multithreaded with moderate allocation; Python and jq mix short- and long-lived objects; and `git status` allocates many per-file structures. On `git status`, `bpf_fault` eliminates the overhead, as most structures are freed without being populated, matching the speculative pattern. ripgrep shows similar behavior. On dense workloads (clang, Python, and jq), `bpf_fault` is close to SIGBUS, as nearly all allocations are touched. Across all applications, `bpf_fault` reduces handler-mode overhead by 1.4–3.9×.

7 Related Work

Applications. `uffd` has been adopted across a range of production and research systems, including post-copy live migration [10], checkpoint/restore [29, 70], the Android Runtime garbage collector [18, 36], tiered memory managers [49, 66, 67, 72], far-memory systems [38, 89], zero-copy userspace I/O [82], stack re-randomization [61], memory debugging [76, 77], and protecting unplugged memory in VMs [35, 40].

No prior work has applied deferred relocation on Linux. Apple’s page-in linking [54] achieves similar lazy semantics but required changes to the Mach-O executable format, while Windows maps libraries at fixed addresses and applies relocations lazily in-kernel [84]. Both approaches require OS-level support that Linux lacks; `bpf_fault` provides equivalent semantics without format or loader changes.

eBPF for memory management. There has been a recent influx of work on using eBPF to customize memory management policies. Much of this work has focused on page cache management, such as `cache_ext` [99, 100], P2Cache [58], PageFlex [90], and FetchBPF [17]. Of these, `bpf_fault` is most similar to `cache_ext`, which uses a `struct_ops` interface for customizing page cache policies entirely in-kernel.

Other works have focused on different aspects of memory management, such as huge page placement [80], page table design [63], and page fault handling. Zussman et al. [98] motivate using eBPF for custom page fault handling, but do not provide a concrete design, implementation, or evaluation. Lightswap [93] implements a user-space swapping system by using eBPF to notify userspace of page faults, but does not push down page fault handling into the kernel. SwiftSweeper and BUDAlloc [3, 4] implement a fault handling hook for secure memory allocators, but do not describe a general-purpose framework or evaluate page fault performance.

Extensible kernels. Custom page fault handling was explored in the extensible kernels of the late 20th century [8, 78, 81]. Mach’s pagers [1, 71] serve page data via IPC, sharing `uffd`’s round-trip overhead. SPIN [12] allows in-kernel fault handlers written in Modula-3, the closest precursor to `bpf_fault`’s approach. Appel and Li [9] cataloged user-level algorithms that benefit from virtual memory primitives. While these designs did not achieve widespread adoption, their ideas influenced both `uffd` and eBPF.

8 Conclusion

This work presents `bpf_fault`, a framework for custom page fault handling in Linux using eBPF. By moving fault handlers into the kernel, `bpf_fault` eliminates the overhead inherent to `uffd`, and enables better application performance, lower resource usage, and novel use cases, such as lazy dynamic linking. Our application integrations demonstrate that `bpf_fault` is practical across diverse use cases, and could be extended to additional systems such as database checkpointing or VM memory protection. More broadly, `bpf_fault` provides an experimentation platform for memory management policies that were previously locked into the kernel. As eBPF’s capabilities continue to expand, so will the range of fault-handling strategies and use cases—for example, per-fault page size selection or application-specific prefaulting.

Acknowledgments

We thank Andrea Arcangeli, Peter Xu, and Nadav Amit (the `userfaultfd()` developers) for their advice; Teng Jiang, Leo Qiao, Frederik Schinner, Chelsea Soemitro, and Shreya Somayajula for their early contributions; Jeremy Carin, Hubertus Franke, and Kostis Kaffes for their feedback; the CloudLab team for their assistance; and the anonymous reviewers. This work was supported by NSF award CNS-2143868. Tal Zussman was supported by NSF award DGE-2036197.

References

- [1] Mike Accetta, Robert Baron, William Bolosky, David Golub, Richard Rashid, Avadis Tevanian, and Michael Young. Mach: A new kernel foundation for UNIX development. In *Proceedings of the Summer 1986 UNIX Conference*, 1986.
- [2] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434, Santa Clara, CA, February 2020. USENIX Association.
- [3] Junho Ahn, Jaehyeon Lee, Kanghyuk Lee, Wooseok Gwak, Minseong Hwang, and Youngjin Kwon. BUDAlloc: Defeating Use-After-Free bugs by decoupling virtual address management from kernel. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 181–197, Philadelphia, PA, August 2024. USENIX Association.
- [4] Junho Ahn, Kanghyuk Lee, Chanyoung Park, Hyungon Moon, and Youngjin Kwon. SwiftSweeper: Defeating use-after-free bugs using memory sweeper without stop-the-world. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 793–809, 2025.
- [5] Daroc Alden. A look at what's possible with BPF arenas. <https://lwn.net/Articles/1019885/>, 2025.
- [6] Daroc Alden. A proposal for shared memory in BPF programs. <https://lwn.net/Articles/961941/>, 2025.
- [7] Daroc Alden. Sending packets directly from BPF, 2026. <https://lwn.net/Articles/1081696/>.
- [8] Thomas E. Anderson, Brian N. Bershad, Edward D. Lazowska, and Henry M. Levy. Scheduler activations: effective kernel support for the user-level management of parallelism. In *Proceedings of the Thirteenth ACM Symposium on Operating Systems Principles, SOSP '91*, pages 95–109, New York, NY, USA, 1991. Association for Computing Machinery.
- [9] Andrew W. Appel and Kai Li. Virtual memory primitives for user programs. In *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS IV*, pages 96–107, New York, NY, USA, 1991. Association for Computing Machinery.
- [10] Andrea Arcangeli. Userland page faults and beyond: Why how and what's next. <https://www.linux-kvm.org/images/1/10/01Wed-1415-LinuxCON-aarcangeli-userfaultfd.pdf>, 2016.
- [11] Andrea Arcangeli and Peter Xu. Meeting with userfaultfd developers, June 2024. Private meeting, June 3, 2024.
- [12] Brian N Bershad, Stefan Savage, Przemyslaw Pardyak, Emin Gün Sirer, Marc E Fluczynski, David Becker, Craig Chambers, and Susan Eggers. Extensibility safety and performance in the SPIN operating system. In *Proceedings of the fifteenth ACM symposium on Operating systems principles*, pages 267–283, 1995.
- [13] Stephen M. Blackburn, Perry Cheng, and Kathryn S. McKinley. Myths and realities: the performance impact of garbage collection. In *Proceedings of the Joint International Conference on Measurement and Modeling of Computer Systems, SIGMETRICS '04/Performance '04*, page 25–36, New York, NY, USA, 2004. Association for Computing Machinery.
- [14] Stephen M. Blackburn, Perry Cheng, and Kathryn S. McKinley. Oil and water? high performance garbage collection in Java with MMTk. In *Proceedings of the 26th International Conference on Software Engineering, ICSE '04*, pages 137–146, USA, 2004. IEEE Computer Society.
- [15] Stephen M. Blackburn, Robin Garner, Chris Hoffmann, Asjad M. Khang, Kathryn S. McKinley, Rotem Bentzur, Amer Diwan, Daniel Feinberg, Daniel Frampton, Samuel Z. Guyer, Martin Hirzel, Antony Hosking, Maria Jump, Han Lee, J. Eliot B. Moss, Aashish Phansalkar, Darko Stefanović, Thomas VanDrunen, Daniel von Dincklage, and Ben Wiedermann. The DaCapo benchmarks: Java benchmarking development and analysis. In *Proceedings of the 21st Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications, OOPSLA '06*, pages 169–190, New York, NY, USA, 2006. Association for Computing Machinery.
- [16] Matthew Butrovich, Samuel Arch, Wan Shen Lim, William Zhang, Jignesh M Patel, and Andrew Pavlo. BPF-DB: A kernel-embedded transactional database management system for eBPF applications. *Proc. ACM Manag. Data*, 3(3), June 2025.
- [17] Xuechun Cao, Shaurya Patel, Soo Yee Lim, Xueyuan Han, and Thomas Pasquier. FetchBPF: Customizable prefetching policies in Linux with eBPF. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 369–378, Santa Clara, CA, July 2024. USENIX Association.
- [18] Seang Chau. Android 13 is in AOSP. <https://android-developers.googleblog.com/2022/08/android-13-is-in-aosp.html>, 2022.
- [19] Christopher Clark, Keir Fraser, Steven Hand, Jacob Gorm Hansen, Eric Jul, Christian Limpach, Ian Pratt, and Andrew Warfield. Live migration of virtual machines. In *Proceedings of the 2nd Conference on Symposium on Networked Systems Design & Implementation - Volume 2, NSDI'05*, pages 273–286, USA, 2005. USENIX Association.
- [20] Containers Project. Default seccomp profile (seccomp.json). GitHub repository file, 2025. Part of the containers/common library, used by Podman and CRI-O.
- [21] Jonathan Corbet. The extensible scheduler class. <https://lwn.net/Articles/922405/>.
- [22] Jonathan Corbet. User-space page fault handling. <https://lwn.net/Articles/636226/>, 2015.
- [23] Jonathan Corbet. Blocking userfaultfd() kernel-fault handling. <https://lwn.net/Articles/819834/>, 2020.
- [24] Jonathan Corbet. BPF for HID drivers, 2022. <https://lwn.net/Articles/909109/>.
- [25] Jonathan Corbet. /dev/userfaultfd. <https://lwn.net/Articles/897260/>, 2022.
- [26] Jonathan Corbet. Custom out-of-memory killers in BPF, 2025. <https://lwn.net/Articles/1019230/>.
- [27] Jonathan Corbet. Improving control over transparent huge page use, 2025. <https://lwn.net/Articles/1032199/>.
- [28] Crispin Cowan, Calton Pu, Dave Maier, Jonathan Walpole, Peat Bakke, Steve Beattie, Aaron Grier, Perry Wagle, Qian Zhang, and Heather Hinton. StackGuard: Automatic adaptive detection and prevention of Buffer-Overflow attacks. In *7th USENIX Security Symposium (USENIX Security 98)*, San Antonio, TX, January 1998. USENIX Association.
- [29] CRIU. <https://criu.org>, 2019.
- [30] Seccomp security profiles for docker. <https://docs.docker.com/engine/security/seccomp>.
- [31] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuangching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. The design and operation of CloudLab. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 1–14, Renton, WA, July 2019. USENIX Association.
- [32] eBPF.io authors. eBPF. <https://ebpf.io/>.
- [33] Jake Edge. The FUSE BPF filesystem, 2023. <https://lwn.net/Articles/937433/>.
- [34] D. R. Engler, M. F. Kaashoek, and J. O'Toole. Exokernel: an operating system architecture for application-level resource management. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles, SOSP '95*, pages 251–266, New York, NY, USA, 1995. Association for Computing Machinery.
- [35] Damian Gasser. Efficient handling of unplugged memory in virtual machine managers. Bachelor's Thesis, Technical University of Munich, March 2025.
- [36] Lokesh Gidra, Hans-J. Boehm, and Joel Fernandes. Utilizing the Linux userfaultfd system call in a compaction phase of a garbage collection process. *Technical Disclosure Commons*, 10 2020.

- [37] Rafael Gonzaga. Potential startup regression since on Node.js >= 22. GitHub Issue, October 2024. Accessed: 2026-03-30.
- [38] Zhiyuan Guo, Zijian He, and Yiyang Zhang. Mira: A program-behavior-guided far memory system. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, pages 692–708, New York, NY, USA, 2023. Association for Computing Machinery.
- [39] Alireza Haddadi, David Black-Schaffer, and Chang Hyun Park. It's not the storage: Exposing Linux kernel bottlenecks in page fault handling on NVMe SSD-backed systems. In *2026 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 125–136, 2026.
- [40] David Hildenbrand and Martin Schulz. virtio-mem: paravirtualized memory hot(un)plug. In *Proceedings of the 17th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, VEE 2021, pages 1–14, New York, NY, USA, 2021. Association for Computing Machinery.
- [41] David Hildenbrand, Martin Schulz, and Nadav Amit. Copy-on-pin: The missing piece for correct copy-on-write. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 2, ASPLOS 2023, pages 176–191, New York, NY, USA, 2023. Association for Computing Machinery.
- [42] Michael Hines, Umesh Deshpande, and Kartik Gopalan. Post-copy live migration of virtual machines. *Operating Systems Review*, 43:14–26, 07 2009.
- [43] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. The eXpress data path: fast programmable packet processing in the operating system kernel. In *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies*, CoNEXT '18, pages 54–66, New York, NY, USA, 2018. Association for Computing Machinery.
- [44] Jann Horn. CVE-2016-4557: Linux: UAF via double-fdput() in bpf(BPF_PROG_LOAD) error path. <https://bugs.chromium.org/p/project-zero/issues/detail?id=3D808>, 2016.
- [45] Amery Huang. Bpf qdisc, 2024. http://oldvger.kernel.org/bpfconf2024_material/BPF-Qdisc.pdf.
- [46] Claire Huang, Stephen Blackburn, and Zixian Cai. Improving garbage collection observability with performance tracing. In *Proceedings of the 20th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes*, MPLR 2023, pages 85–99, New York, NY, USA, 2023. Association for Computing Machinery.
- [47] Jack Tigar Humphries, Neel Natsu, Ashwin Chaugule, Ofir Weisse, Barret Rhoden, Josh Don, Luigi Rizzo, Oleg Rombakh, Paul Turner, and Christos Kozyrakis. GhOST: Fast & flexible user-space delegation of Linux scheduling. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, SOSP '21, pages 588–604, New York, NY, USA, 2021. Association for Computing Machinery.
- [48] Jaeseong Im, Jongyul Kim, Youngjin Kwon, and Seungryoul Maeng. On-demand virtualization for post-copy OS migration in bare-metal cloud. *IEEE Transactions on Cloud Computing*, PP:1–1, 01 2022.
- [49] Sepehr Jalalian, Shaurya Patel, Milad Rezaei Hajidehi, Margo Seltzer, and Alexandra Fedorova. ExtMem: Enabling Application-Aware virtual memory management for Data-Intensive applications. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 397–408, Santa Clara, CA, July 2024. USENIX Association.
- [50] Jinghao Jia, YiFei Zhu, Dan Williams, Andrea Arcangeli, Claudio Canella, Hubertus Franke, Tobin Feldman-Fitzthum, Dimitrios Skarlatos, Daniel Gruss, and Tianyin Xu. Programmable system call security with eBPF, 2023.
- [51] M. Frans Kaashoek, Dawson R. Engler, Gregory R. Ganger, Hector M. Briceño, Russell Hunt, David Mazières, Thomas Pinckney, Robert Grimm, John Jannotti, and Kenneth Mackenzie. Application Performance and Flexibility on Exokernel Systems. In *Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles*, SOSP '97, pages 52–65, New York, NY, USA, 1997. Association for Computing Machinery.
- [52] Kostis Kaffes, Jack Tigar Humphries, David Mazières, and Christos Kozyrakis. Syrup: User-defined scheduling across the stack. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, SOSP '21, pages 605–620, New York, NY, USA, 2021. Association for Computing Machinery.
- [53] Haim Kermany and Erez Petrank. The compressor: concurrent, incremental, and parallel compaction. *SIGPLAN Not.*, 41(6):354–363, June 2006.
- [54] Nick Kledzik. Link fast: Improve build and launch times. <https://developer.apple.com/videos/play/wwdc2022/110362/>, 2022. WWDC 2022, Session 110362.
- [55] KVM Community. KVM migration. <https://www.linux-kvm.org/page/Migration>. Accessed: 2026-03-31.
- [56] Martin Lau. BPF extensible network, 2020. https://lpc.events/event/7/contributions/687/attachments/537/1262/BPF_network_tcp-cc-hdr-sk-stg_LPC_2020.pdf.
- [57] Martin KaFai Lau. struct-ops, 2020. <https://lwn.net/Articles/809092/>.
- [58] Dusol Lee, Inhyuk Choi, Chanyoung Lee, Hyungsoo Jung, and Jihong Kim. P2Cache: Enhancing data-centric applications via application-guided management of OS page caches. *ACM Trans. Storage*, June 2025. Just Accepted.
- [59] Zhenpeng Lin, Yuhang Wu, and Xinyu Xing. Dirtycred: Escalating privilege in Linux kernel. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, CCS '22, pages 1963–1976, New York, NY, USA, 2022. Association for Computing Machinery.
- [60] Denis V. Lunev. Towards asynchronous revert in QEMU, 2021. https://static.sched.com/hosted_files/kvmforum2021/7a/kvm2021-background-snapshot.pdf.
- [61] Robert Lyerly, Xiaoguang Wang, and Binoy Ravindran. Dynamic and secure memory transformation in userspace. In *Computer Security – ESORICS 2020: 25th European Symposium on Research in Computer Security*, ESORICS 2020, Guildford, UK, September 14–18, 2020, *Proceedings, Part I*, pages 237–256, Berlin, Heidelberg, 2020. Springer-Verlag.
- [62] Sebastiano Miano, Matteo Bertrone, Fulvio Rizzo, Mauricio Vásquez Bernal, Yunsong Lu, and Jianwen Pi. Securing Linux with a faster and scalable iptables. *SIGCOMM Comput. Commun. Rev.*, 49(3):2–17, November 2019.
- [63] Konstantinos Mores, Stratos Psomadakis, and Georgios Goumas. eBPF-mm: Userspace-guided memory management in Linux with eBPF, 2024.
- [64] Vitaly Nikolenko. CVE-2016-6187: Exploiting Linux kernel heap off-by-one. <https://duasynt.com/blog/cve-2016-6187-heap-off-by-one-exploit>, 2016.
- [65] Vitaly Nikolenko. Linux Kernel universal heap spray. <https://duasynt.com/blog/linux-kernel-heap-spray>, 2018.
- [66] Ivy Peng, Marty McFadden, Eric Green, Keita Iwabuchi, Kai Wu, Dong Li, Roger Pearce, and Maya Gokhale. UMap: Enabling application-driven optimizations for page management. In *2019 IEEE/ACM Workshop on Memory Centric High Performance Computing (MCHPC)*, pages 71–78, 2019.
- [67] Ivy B. Peng, Maya B. Gokhale, Karim Youssef, Keita Iwabuchi, and Roger Pearce. Enabling scalable and extensible memory-mapped datastores in userspace. *IEEE Transactions on Parallel and Distributed Systems*, 33(4):866–877, 2022.
- [68] Bruce Perens. libefence(3) - Linux man page. <https://linux.die.net/man/3/libefence>. efence - Electric Fence Malloc Debugger.
- [69] Stratos Psomadakis, Dimitrios Siakavaras, Chloe Alverti, Symeon Porgiotis, Orestis Lagkas Nikolas, Christos Katsakioris, Konstantinos Nikas, Georgios Goumas, and Nectarios Koziris. SnapBPF: Exploiting eBPF for serverless snapshot prefetching. In *Proceedings of the 17th ACM Workshop on Hot Topics in Storage and File Systems*, HotStorage

- '25, pages 9–16, New York, NY, USA, 2025. Association for Computing Machinery.
- [70] Mike Rapoport. Userfaultfd: Post-copy VM migration and beyond. https://blog.linuxplumbersconf.org/2017/ocw/system/presentations/4699/original/userfaultfd_%20post-copy%20VM%20migration%20and%20beyond.pdf, 2017.
 - [71] Richard Rashid, Avadis Tevanian, Michael Young, David Golub, Robert Baron, David Black, William Bolosky, and Jonathan Chew. Machine-independent virtual memory management for paged uniprocessor and multiprocessor architectures. *SIGOPS Oper. Syst. Rev.*, 21(4):31–39, October 1987.
 - [72] Amanda Raybuck, Tim Stamler, Wei Zhang, Mattan Erez, and Simon Peter. HeMem: Scalable tiered memory management for big data applications and real NVM. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, SOSP '21, pages 392–407, New York, NY, USA, 2021. Association for Computing Machinery.
 - [73] Redis. memtier_benchmark: A high-throughput benchmarking tool for Redis & Memcached. https://redis.io/blog/memtier_benchmark-a-high-throughput-benchmarking-tool-for-redis-memcached/, 2013.
 - [74] Adam Ruprecht, Danny Jones, Dmitry Shiraev, Greg Harmon, Maya Spivak, Michael Krebs, Miche Baker-Harvey, and Tyler Sanderson. VM live migration at scale. *SIGPLAN Not.*, 53(3):45–56, Mar 2018.
 - [75] Marta Ryczyńska. Introducing maple trees. <https://lwn.net/Articles/845507/>, 2021.
 - [76] Frederik Schinner. Efficient electric fences using userfaultfd. Bachelor's Thesis, Technical University of Munich, September 2024.
 - [77] Frederik Schinner and David Hildenbrand. efency: An efficient electric-fence malloc() debugger for the 21st century, 2024. <https://gitlab.com/efency/efency>.
 - [78] Margo I. Seltzer, Yasuhiro Endo, Christopher Small, and Keith A. Smith. Dealing with disaster: Surviving misbehaved kernel extensions. In *Proceedings of the Second USENIX Symposium on Operating Systems Design and Implementation*, OSDI '96, pages 213–227, New York, NY, USA, 1996. Association for Computing Machinery.
 - [79] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. AddressSanitizer: A fast address sanity checker. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, pages 309–318, Boston, MA, June 2012. USENIX Association.
 - [80] Dimitrios Skarlatos and Kaiyang Zhao. Towards programmable memory management with eBPF, 2024. [https://lpc.events/event/18/contributions/1932/attachments/1646/3414/Towards%20Programmable%20Memory%20Management%20with%20eBPF%20\(LPC%202024\).pdf](https://lpc.events/event/18/contributions/1932/attachments/1646/3414/Towards%20Programmable%20Memory%20Management%20with%20eBPF%20(LPC%202024).pdf).
 - [81] Christopher A Small and Margo I Seltzer. Vino: An integrated platform for operating system and database research. *Harvard Computer Science Group Technical Report TR-30-94*, 1994.
 - [82] Timothy Stamler, Deukyeon Hwang, Amanda Raybuck, Wei Zhang, and Simon Peter. zIO: Accelerating IO-Intensive applications with transparent Zero-Copy IO. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 431–445, Carlsbad, CA, July 2022. USENIX Association.
 - [83] The Chromium Authors. Linux zygote. <https://chromium.googlesource.com/chromium/src/+refs/heads/main/docs/linux/zygote.md>, 2024.
 - [84] The Chromium Authors. Native relocations. https://chromium.googlesource.com/chromium/src/+master/docs/native_relocations.md, 2024.
 - [85] Ives van Hoorne. Cloning microVMs by sharing memory through userfaultfd, 2023. <https://codesandbox.io/blog/cloning-microvms-using-userfaultfd>.
 - [86] David Vernet. More flexible memory access for BPF programs, 2022. <https://lwn.net/Articles/910873/>.
 - [87] David Vernet. NUMA rebalancing on tiered-memory systems. LWN.net, May 2022.
 - [88] Albert Mingkun Yang and Tobias Wrigstad. Deep dive into ZGC: A modern garbage collector in OpenJDK. *ACM Trans. Program. Lang. Syst.*, 44(4), September 2022.
 - [89] Anil Yelam, Stewart Grant, Saarth Deshpande, Nadav Amit, Radhika Niranjana Mysore, Amy Ousterhout, Marcos K. Aguilera, and Alex C. Snoeren. Eden: Developer-Friendly Application-Integrated far memory. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 1067–1083, Philadelphia, PA, April 2025. USENIX Association.
 - [90] Anil Yelam, Kan Wu, Zhiyuan Guo, Suli Yang, Rajath Shashidhara, Wei Xu, Stanko Novaković, Alex C Snoeren, and Kimberly Keeton. PageFlex: Flexible and efficient user-space delegation of Linux paging policies with eBPF. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pages 291–306, 2025.
 - [91] Ioannis Zarkadas, Tal Zussman, Jeremy Carin, Sheng Jiang, Yuhong Zhong, Jonas Pfefferle, Hubertus Franke, Junfeng Yang, Kostis Kaffes, Ryan Stutsman, and Asaf Cidon. BPF-oF: Storage function pushdown over the network, 2023.
 - [92] Yusheng Zheng, Tong Yu, Yiwei Yang, Minghui Jiang, Xiangyu Gao, Jianchang Su, Yanpeng Hu, Wenan Mao, Wei Zhang, Dan Williams, et al. gpu_ext: Extensible OS policies for GPUs via eBPF. *arXiv preprint arXiv:2512.12615*, 2025.
 - [93] Kan Zhong, Wenlin Cui, Xin Chen, Qiao Li, Zhe Yang, Youyou Lu, Xiaodan Yan, Siwei Luo, Qizhao Yuan, and Keji Huang. Revisiting swapping in user-space with lightweight threading. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42:4205–4218, 2021.
 - [94] Yuhong Zhong, Haoyu Li, Yu Jian Wu, Ioannis Zarkadas, Jeffrey Tao, Evan Mesterhazy, Michael Makris, Junfeng Yang, Amy Tai, Ryan Stutsman, and Asaf Cidon. XRP: In-Kernel storage functions with eBPF. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 375–393, Carlsbad, CA, July 2022. USENIX Association.
 - [95] Yang Zhou, Zezhou Wang, Sowmya Dharanipragada, and Minlan Yu. Electrode: Accelerating distributed protocols with eBPF. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1391–1407, Boston, MA, April 2023. USENIX Association.
 - [96] Yang Zhou, Xingyu Xiang, Matthew Kiley, Sowmya Dharanipragada, and Minlan Yu. DINT: Fast In-Kernel distributed transactions with eBPF. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 401–417, Santa Clara, CA, April 2024. USENIX Association.
 - [97] Ori Ben Zur, Jakob Krebs, Shai Aviram Bergman, and Mark Silberstein. Accelerating nested virtualization with HyperTurtle. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference*, USENIX ATC '25, USA, 2025. USENIX Association.
 - [98] Tal Zussman, Teng Jiang, and Asaf Cidon. Custom page fault handling with eBPF. In *Proceedings of the ACM SIGCOMM 2024 Workshop on eBPF and Kernel Extensions*, eBPF '24, pages 71–73, New York, NY, USA, 2024. Association for Computing Machinery.
 - [99] Tal Zussman, Ioannis Zarkadas, Jeremy Carin, Andrew Cheng, Hubertus Franke, Jonas Pfefferle, and Asaf Cidon. cache_ext: Customizing the page cache with eBPF. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, SOSP '25, pages 462–478, New York, NY, USA, 2025. Association for Computing Machinery.
 - [100] Tal Zussman, Ioannis Zarkadas, Jeremy Carin, Andrew Cheng, Hubertus Franke, Jonas Pfefferle, and Asaf Cidon. cache_ext: Customizing and tracing the page cache with eBPF. *ACM Trans. Comput. Syst.*, June 2026. Just Accepted.